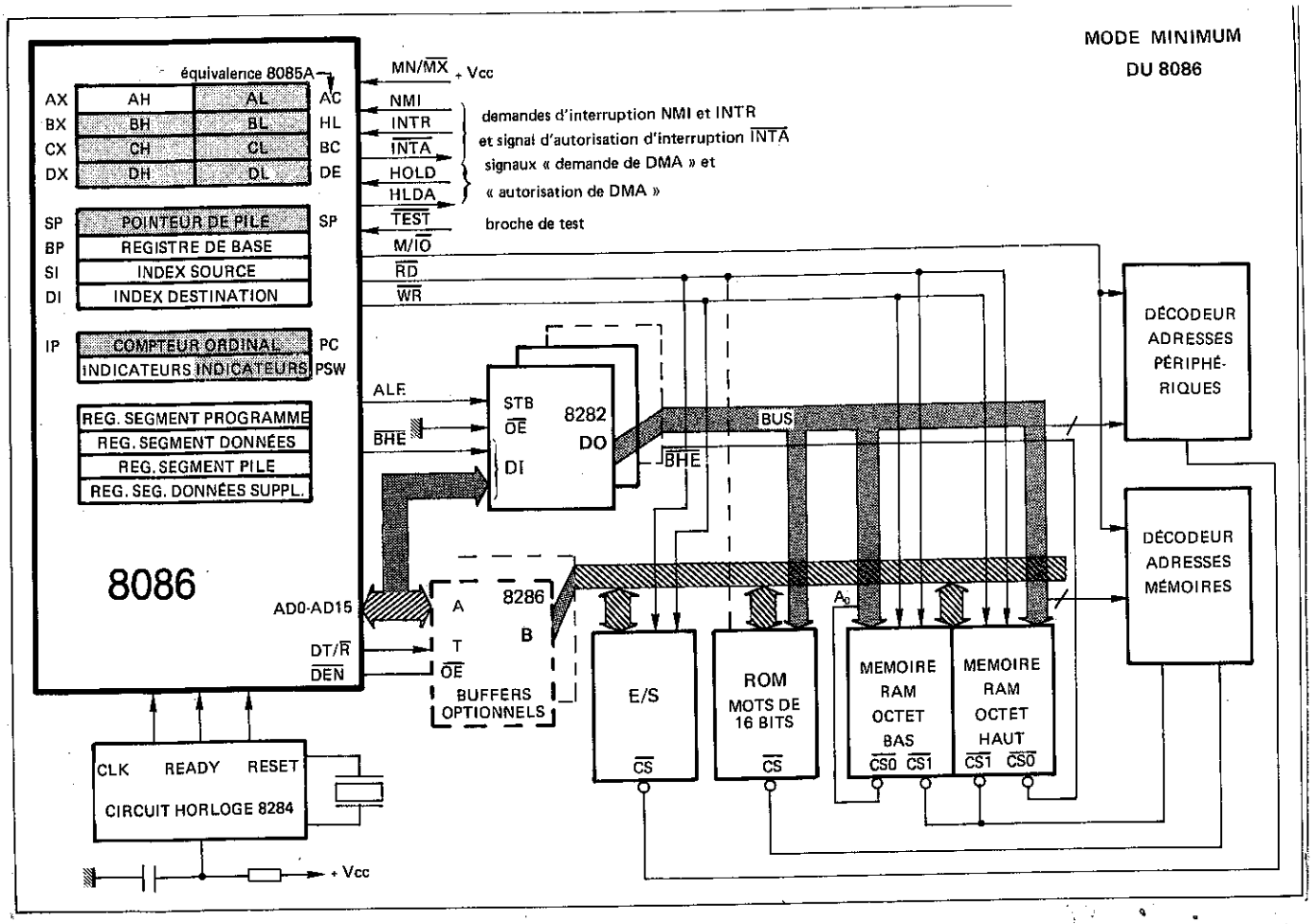
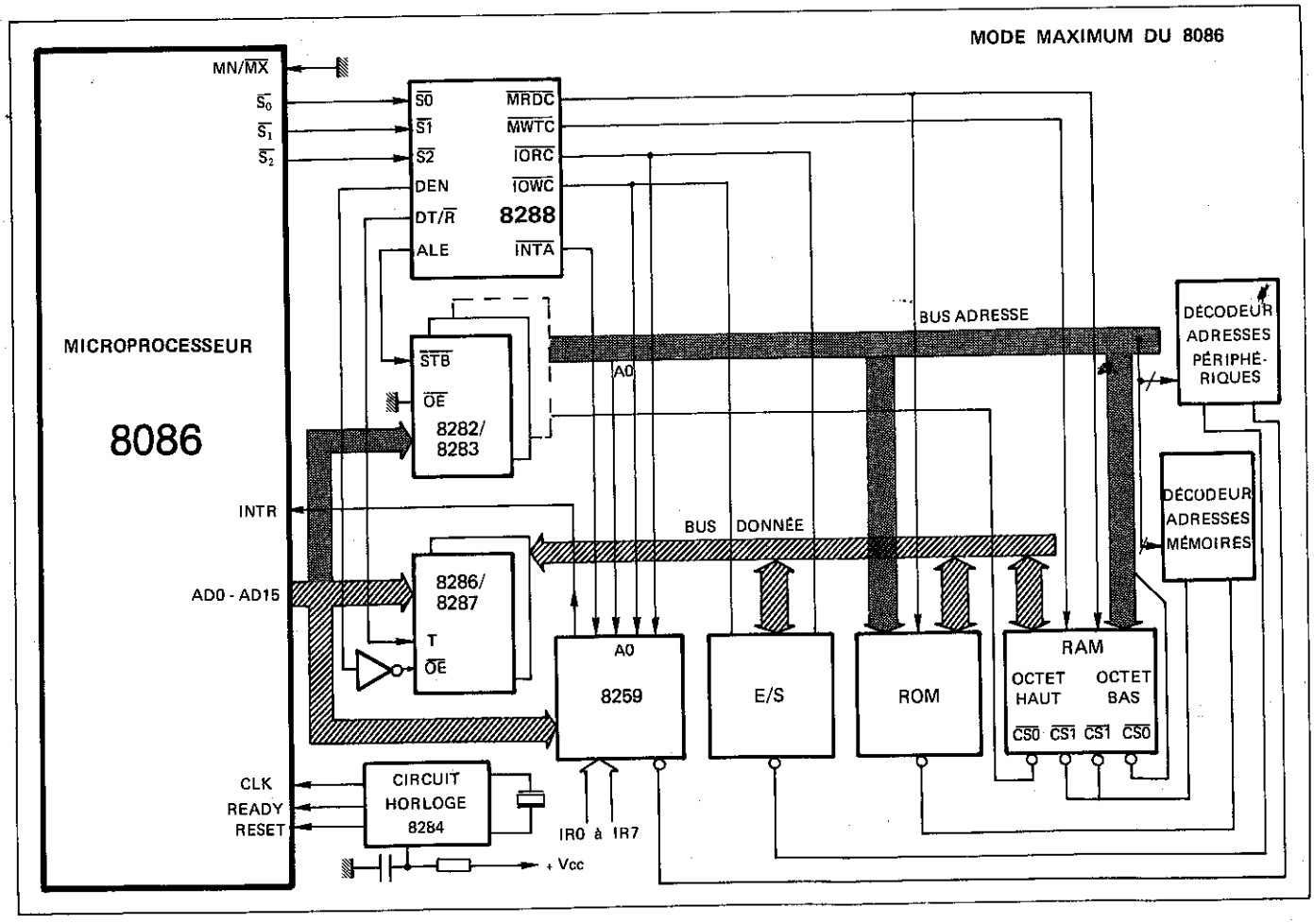
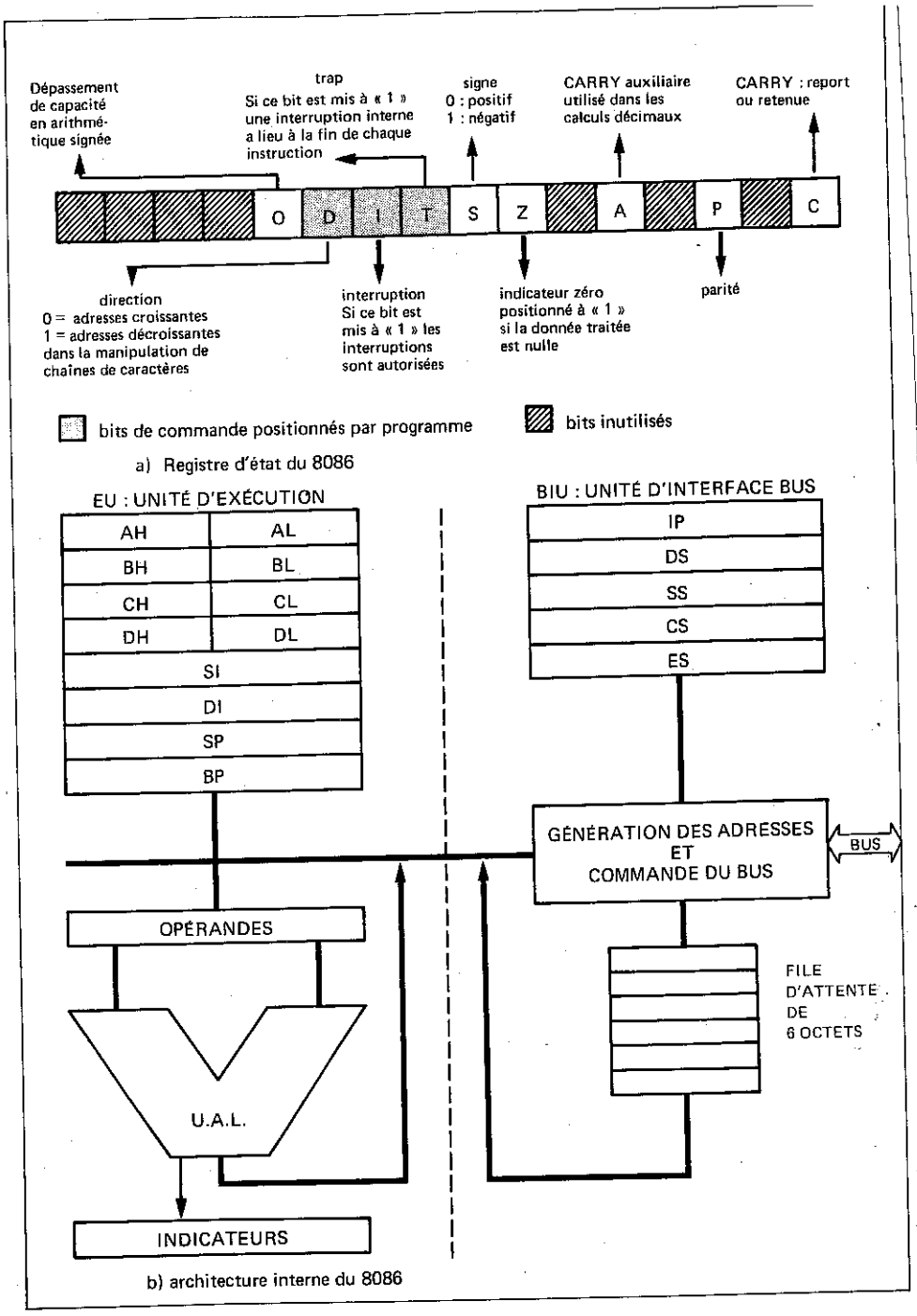


Organisation externe du 8086 en mode : minimum

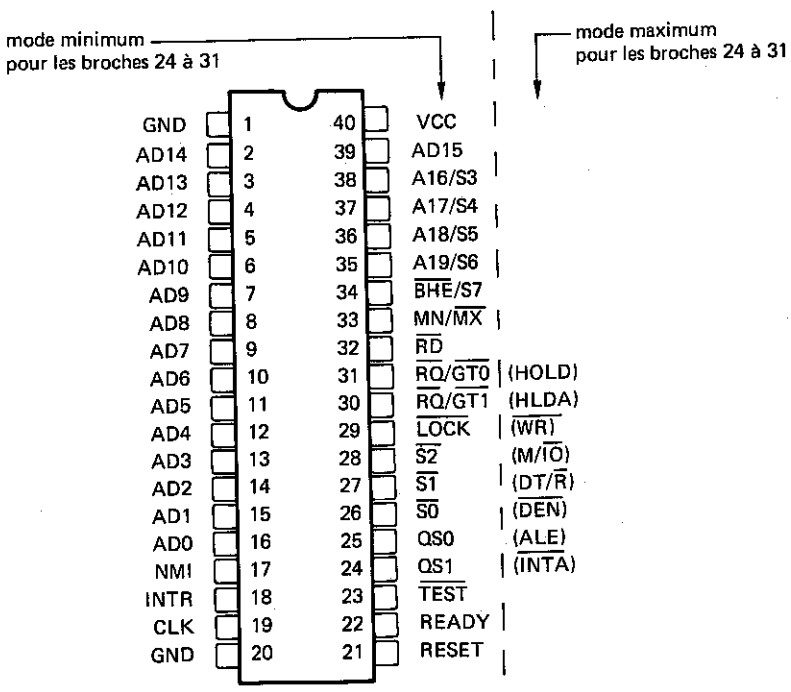
maximum



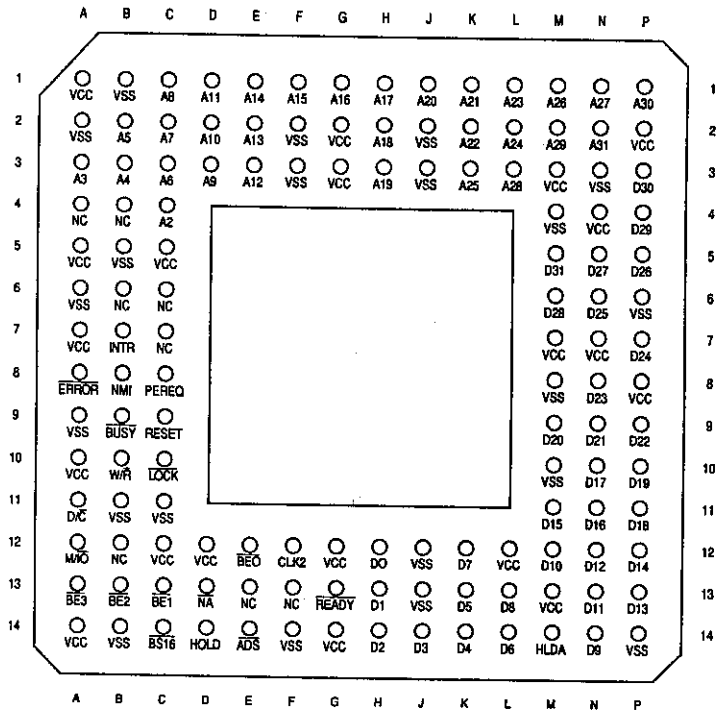
Architecture interne du 8086.



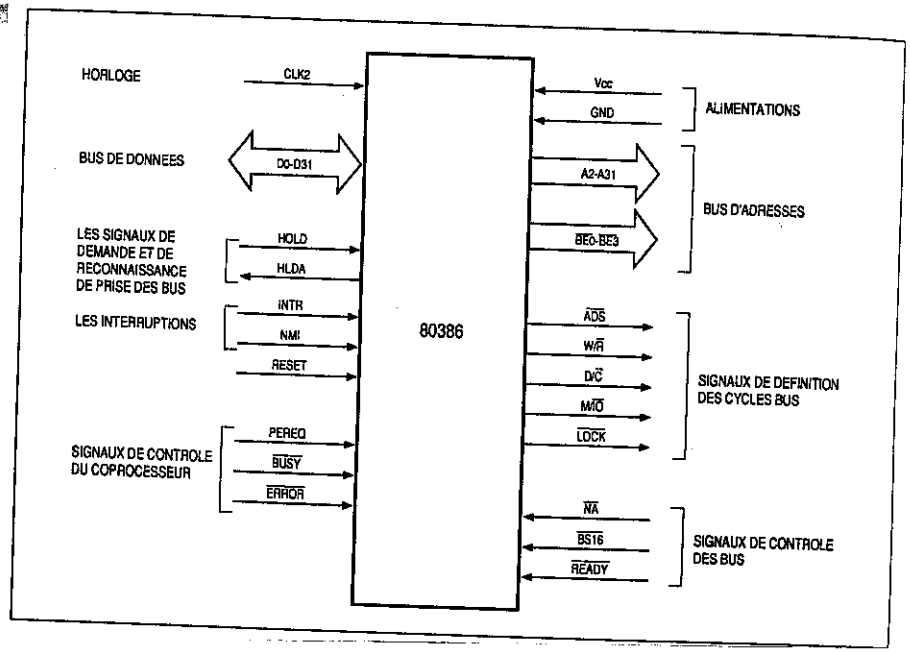
Brochage du 8086

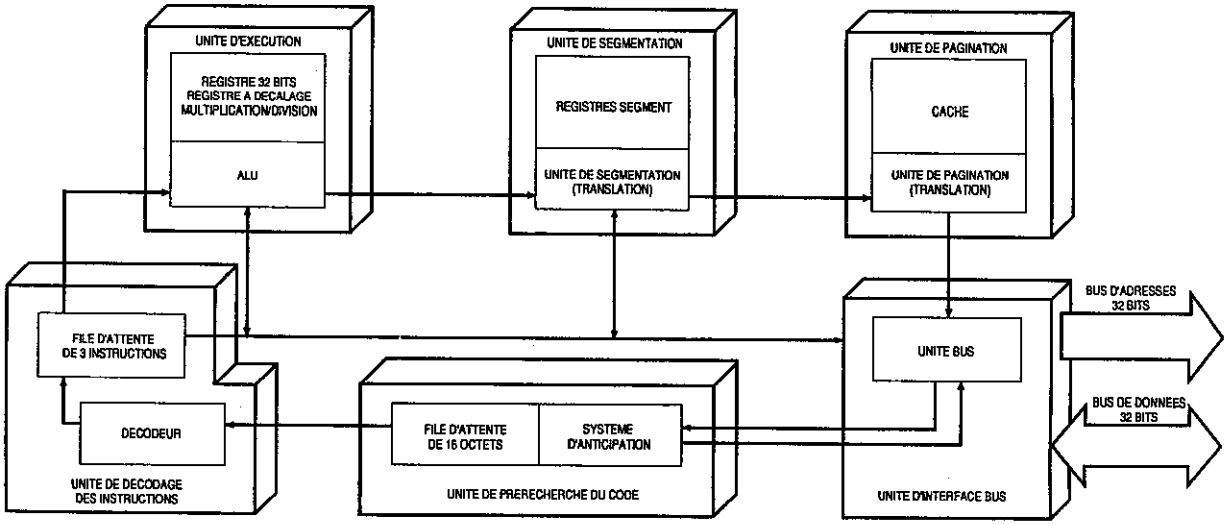


Le brochage du microprocesseur 80386 apparaît sur la Figure 1.1. Ce boîtier de type *pin grid array* possède 132 broches dont 124 sont utilisées. Ce processeur est implémenté en technologie CMOS par le procédé Intel CHMOS-III, il intègre 275 000 transistors sur une surface de 1 cm².



NC : non connecté





Signaux d'état du 8086

$\overline{S_2}$	$\overline{S_1}$	$\overline{S_0}$	TYPE DE TRANSFERT EN COURS
0	0	0	autorisation d'une interruption
0	0	1	lecture périphérique
0	1	0	écriture périphérique
0	1	1	halte
1	0	0	recherche d'une instruction
1	0	1	lecture mémoire
1	1	0	écriture mémoire
1	1	1	aucun

S_4	S_3	SEGMENT SÉLECTIONNÉ
0	0	« données supplémentaires »
0	1	« pile »
1	0	« programme »
1	1	« donnée »

- S_5 est une copie du bit d'état « interruption » I.
- S_6 est au niveau zéro
- S_7 est un bit d'état inutilisé mais disponible pour une éventuelle version du 8086

QS_1	QS_0	ÉTAT DE LA FILE D'ATTENTE
0	0	Aucune opération en cours
1	0	Le premier octet d'une instruction a été prélevé dans la file d'attente
0	1	La file d'attente a été réinitialisée
1	1	L'octet suivant d'une instruction a été prélevé dans la file d'attente

Signaux extérieurs du 8086

SIGNAUX COMMUNS AUX DEUX MODES DU 8086

- • AD_0 à AD_{15} bus adresse / donnée multiplexé
- • AD_{16}/S_3, A_{17}/S_4, A_{18}/S_5 et A_{19}/S_6. Sur ces quatre sorties multiplexées le μP délivre les quatre bits poids forts de l'adresse puis des signaux d'état
- • $\overline{BHE}/S_7$. Sur cette sortie multiplexée le μP délivre le signal $\overline{BHE}$ validant l'octet poids fort de la donnée puis un signal d'état
- • $\overline{RD}$ signal de lecture
- • READY synchronisation avec les mémoires ou les périphériques lents.
- • CLK entrée horloge du 8086
- • RESET initialisation du 8086
- • NMI demande d'interruption non masquable active sur front montant
- • INTR demande d'interruption masquable active au niveau haut
- • $\overline{TEST}$ synchronisation avec un processeur spécialisé

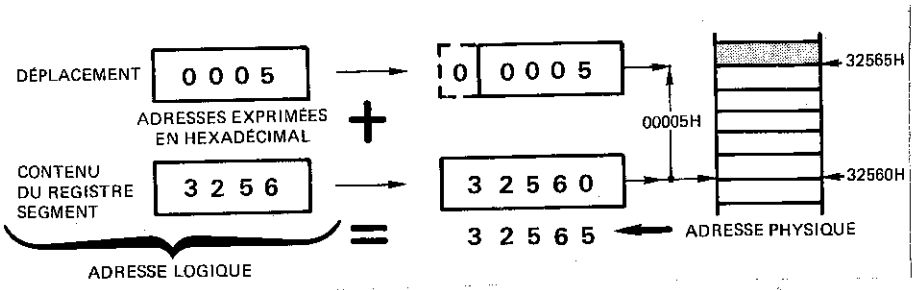
SIGNAUX SPECIFIQUES AU MODE MINIMUM	SIGNAUX SPECIFIQUES AU MODE MAXIMUM
• • $M/\overline{IO}$ sélection d'une mémoire ($M/\overline{IO} = 1$) ou d'un périphérique ($M/\overline{IO} = 0$) • • $\overline{WR}$ signal d'écriture • • $\overline{INTA}$ autorisation d'interruption • • DT/$\overline{R}$ } signaux nécessaires aux circuits • • $\overline{DEN}$ } buffers du bus donnée • ALE signal de verrouillage de l'adresse • HOLD demande d'accès direct mémoire • HLDA autorisation d'accès direct mémoire	• • $\overline{S_0}$, $\overline{S_1}$, $\overline{S_2}$ signaux d'état décodés par le 8288 • $\overline{RQ}/\overline{GT_0}$, $\overline{RQ}$, $\overline{GT_1}$ demandes et autorition d'accès au bus • • $\overline{LOCK}$ interdiction d'accès au bus en fonctionnement multiprocesseur • QS_0, QS_1 état de la file d'attente

le mode minimum et maximum

CARACTÉRISTIQUES COMPARÉES DES DEUX MODES DU 8086

	MODE MINIMUM	MODE MAXIMUM
connexion du signal MN/$\overline{MX}$	au + 5 V	à la masse
circuits constituant le processeur	8284 + 8086 + 2 x 8282	8284 + 8086 + 8288 + 3 x 8282
capacité d'adressage maximale	1 Mégaoctets en théorie 32 ou 64 K le plus souvent	1 Mégaoctets
adressage des ports d'E/S	par instructions mémoire ou par instructions E/S en théorie mais par instructions E/S le plus souvent	par instructions mémoire ou par instructions E/S
nombre de ports d'E/S adressables par instructions E/S	64 K ports de 8 bits en indirect par DX avec AL ou 32 K ports de 16 bits en indirect par DX avec AX ou 256 ports de 8 bits en direct avec AL ou 128 ports de 8 bits en direct avec AX	

Remarque : Les signaux précédés de deux points sont en logique trois états

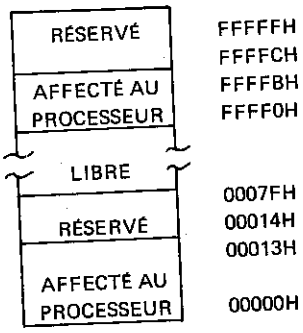


Traduction adresse logique en adresse physique

type d'accès mémoire	registre segment pris par défaut	autre registre segment possible	registre contenant le déplacement
1. Recherche de l'instruction (fetch)	CS	aucun	IP
2. Opération sur la pile	SS	aucun	SP
3. Accès à une donnée sauf 4, 5, 6	DS	CS,ES,SS	adresse effective
4. Accès à la donnée source } d'une chaîne de caractères	DS	CS,ES,SS	SI
5. Accès à la donnée destination } d'une chaîne de caractères	ES	aucun	DI
6. Accès à une donnée avec BP comme registre de base	SS	CS,DS,ES	adresse effective

Registres segment définis implicitement par le type d'accès mémoire

Positions mémoires réservées



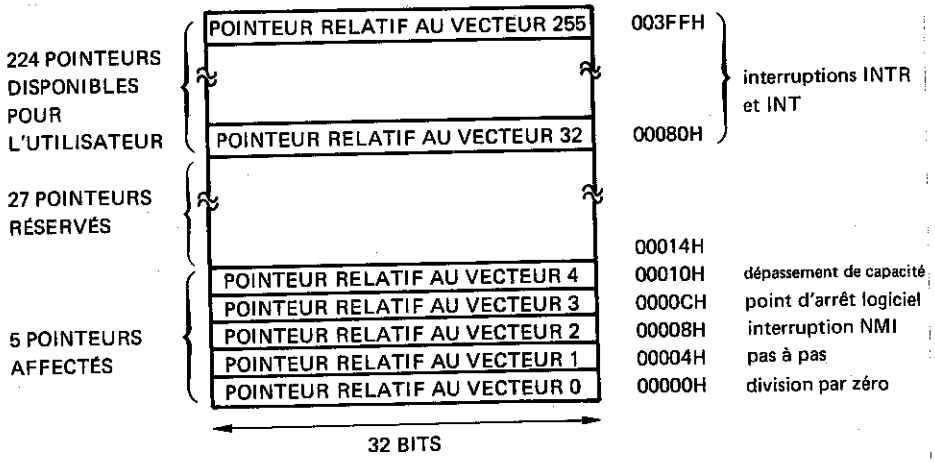


Table des vecteurs d'Interruptions

JEU D'INSTRUCTIONS DU 8086/8088 DONNÉES DE RÉFÉRENCE

AAA

AAA (no operands)
ASCII adjust for addition

Flags: O D I T S Z A P C
U U U X U X

Operands	Clocks	Transfers*	Bytes	Coding Example
(no operands)	4	—	1	AAA

AAD

AAD (no operands)
ASCII adjust for division

Flags: O D I T S Z A P C
U X X U X U

Operands	Clocks	Transfers*	Bytes	Coding Example
(no operands)	60	—	2	AAD

AAM

AAM (no operands)
ASCII adjust for multiply

Flags: O D I T S Z A P C
U X X U X U

Operands	Clocks	Transfers*	Bytes	Coding Example
(no operands)	83	—	1	AAM

AAS

AAS (no operands)
ASCII adjust for subtraction

Flags: O D I T S Z A P C
U U U X U X

Operands	Clocks	Transfers*	Bytes	Coding Example
(no operands)	4	—	1	AAS

ADC

ADC destination,source
Add with carry

Flags: O D I T S Z A P C
X X X X X X

Operands	Clocks	Transfers*	Bytes	Coding Example
register, register	3	—	2	ADC AX, SI
register, memory	9+EA	1	2-4	ADC DX, BETA [SI]
memory, register	16+EA	2	2-4	ADC ALPHA [BX] [SI], DI
register, immediate	4	—	3-4	ADC BX, 256
memory, immediate	17+EA	2	3-6	ADC GAMMA, 30H
accumulator, immediate	4	—	2-3	ADC AL, 5

ADD

ADD destination,source
Addition

Flags: O D I T S Z A P C
X X X X X X

Operands	Clocks	Transfers*	Bytes	Coding Example
register, register	3	—	2	ADD CX, DX
register, memory	9+EA	1	2-4	ADD DI, [BX].ALPHA
memory, register	16+EA	2	2-4	ADD TEMP, CL
register, immediate	4	—	3-4	ADD CL, 2
memory, immediate	17+EA	2	3-6	ADD ALPHA, 2
accumulator, immediate	4	—	2-3	ADD AX, 200

AND

AND destination,source
Logical and

Flags: O D I T S Z A P C
0 X X U X 0

Operands	Clocks	Transfers*	Bytes	Coding Example
register, register	3	—	2	AND AL,BL
register, memory	9+EA	1	2-4	AND CX,FLAG_WORD
memory, register	16+EA	2	2-4	AND ASCII [DI],AL
register, immediate	4	—	3-4	AND CX,0F0H
memory, immediate	17+EA	2	3-6	AND BETA, 01H
accumulator, immediate	4	—	2-3	AND AX, 01010000B

CALL

CALL target
Call a procedure

Flags: O D I T S Z A P C

Operands	Clocks	Transfers*	Bytes	Coding Examples
near-proc	19	1	3	CALL NEAR_PROC
far-proc	28	2	5	CALL FAR_PROC
memptr 16	21+EA	2	2-4	CALL PROC_TABLE [SI]
regptr 16	16	1	2	CALL AX
memptr 32	37+EA	4	2-4	CALL [BX].TASK [SI]

CBW

CBW (no operands)
Convert byte to word

Flags: O D I T S Z A P C

Operands	Clocks	Transfers*	Bytes	Coding Example
(no operands)	2	—	1	CBW

CLC

CLC (no operands)
Clear carry flag

Flags: O D I T S Z A P C
0

Operands	Clocks	Transfers*	Bytes	Coding Example
(no operands)	2	—	1	CLC

CLD

CLD (no operands)
Clear direction flag

Flags: O D I T S Z A P C
0

Operands	Clocks	Transfers*	Bytes	Coding Example
(no operands)	2	—	1	CLD

CLI

CLI (no operands)
Clear interrupt flag

Flags: O D I T S Z A P C
0

Operands	Clocks	Transfers*	Bytes	Coding Example
(no operands)	2	—	1	CLI

CMC

CMC (no operands)
Complement carry flag

Flags: O D I T S Z A P C
X

Operands	Clocks	Transfers*	Bytes	Coding Example
(no operands)	2	—	1	CMC

CMP

CMP destination,source
Compare destination to source

Flags	O	D	I	T	S	Z	A	P	C
	X				X	X	X	X	X

Operands	Clocks	Transfers*	Bytes	Coding Example
register, register	3	—	2	CMP BX, CX
register, memory	9 + EA	1	2-4	CMP DH, ALPHA
memory, register	9 + EA	1	2-4	CMP [BP + 2], SI
register, immediate	4	—	3-4	CMP BL, 02H
memory, immediate	10 + EA	1	3-6	CMP [BX].RADAR [DI], 3420H
accumulator, immediate	4	—	2-3	CMP AL, 00010000B

CMPS

CMPS dest-string,source-string
Compare string

Flags	O	D	I	T	S	Z	A	P	C
	X				X	X	X	X	X

Operands	Clocks	Transfers*	Bytes	Coding Example
dest-string, source-string	22	2	1	CMPS BUFF1, BUFF2
(repeat) dest-string, source-string	9 + 22/rep	2/rep	1	REPE CMPS ID, KEY

CWD

CWD (no operands)
Convert word to doubleword

Flags	O	D	I	T	S	Z	A	P	C

Operands	Clocks	Transfers*	Bytes	Coding Example
(no operands)	5	—	1	CWD

DAA

DAA (no operands)
Decimal adjust for addition

Flags	O	D	I	T	S	Z	A	P	C
	X				X	X	X	X	X

Operands	Clocks	Transfers*	Bytes	Coding Example
(no operands)	4	—	1	DAA

DAS

DAS (no operands)
Decimal adjust for subtraction

Flags	O	D	I	T	S	Z	A	P	C
	U				X	X	X	X	X

Operands	Clocks	Transfers*	Bytes	Coding Example
(no operands)	4	—	1	DAS

DEC

DEC destination
Decrement by 1

Flags	O	D	I	T	S	Z	A	P	C
	X				X	X	X	X	

Operands	Clocks	Transfers*	Bytes	Coding Example
reg16	2	—	1	DEC AX
reg8	3	—	2	DEC AL
memory	15 + EA	2	2-4	DEC ARRAY [SI]

DIV

DIV source
Division, unsigned

Flags	O	D	I	T	S	Z	A	P	C
	U				U	U	U	U	U

Operands	Clocks	Transfers*	Bytes	Coding Example
reg8	80-90	—	2	DIV CL
reg16	144-162	—	2	DIV BX
mem8	(86-96) + EA	1	2-4	DIV ALPHA
mem16	(150-168) + EA	1	2-4	DIV TABLE [SI]

ESC

ESC external-opcode,source
Escape

Flags	O	D	I	T	S	Z	A	P	C

Operands	Clocks	Transfers*	Bytes	Coding Example
immediate, memory	8 + EA	1	2-4	ESC 6,ARRAY [SI]
immediate, register	2	—	2	ESC 20,AL

HLT

HLT (no operands)
Halt

Flags	O	D	I	T	S	Z	A	P	C

Operands	Clocks	Transfers*	Bytes	Coding Example
(no operands)	2	—	1	HLT

IDIV

IDIV source
Integer division

Flags	O	D	I	T	S	Z	A	P	C
	U				U	U	U	U	U

Operands	Clocks	Transfers*	Bytes	Coding Example
reg8	101-112	—	2	IDIV BL
reg16	165-184	—	2	IDIV CX
mem8	(107-118) + EA	1	2-4	IDIV DIVISOR_BYTE [SI]
mem16	(171-190) + EA	1	2-4	IDIV [BX].DIVISOR_WORD

IMUL

IMUL source
Integer multiplication

Flags	O	D	I	T	S	Z	A	P	C
	X				U	U	U	U	X

Operands	Clocks	Transfers*	Bytes	Coding Example
reg8	80-98	—	2	IMUL CL
reg16	128-154	—	2	IMUL BX
mem8	(86-104) + EA	1	2-4	IMUL RATE BYTE
mem16	(134-160) + EA	1	2-4	IMUL RATE WORD [BP] [DI]

IN

IN accumulator,port
Input byte or word

Flags	O	D	I	T	S	Z	A	P	C

Operands	Clocks	Transfers*	Bytes	Coding Example
accumulator, immed8	10	1	2	IN AL, 0FFEAH
accumulator, DX	8	1	1	IN AX, DX

INC

INC destination
Increment by 1

Flags	O	D	I	T	S	Z	A	P	C
	X				X	X	X	X	

Operands	Clocks	Transfers*	Bytes	Coding Example
reg16	2	—	1	INC CX
reg8	3	—	2	INC BL
memory	15 + EA	2	2-4	INC ALPHA [DI] [BX]

INT

INT interrupt-type
Interrupt

Flags	O	D	I	T	S	Z	A	P	C
			0	0					

Operands	Clocks	Transfers*	Bytes	Coding Example
immed8 (type = 3)	52	5	1	INT 3
immed8 (type ≠ 3)	51	5	2	INT 67

INTR†	INTR (external maskable interrupt) Interrupt if INTR and IF=1			Flags O D I T S Z A P C 0 0
Operands	**Clocks**	**Transfers***	**Bytes**	**Coding Example**
(no operands)	61	7	N/A	N/A

INTO	INTO (no operands) Interrupt if overflow			Flags O D I T S Z A P C 0 0
Operands	**Clocks**	**Transfers***	**Bytes**	**Coding Example**
(no operands)	53 or 4	5	1	INTO

IRET	IRET (no operands) Interrupt Return			Flags O D I T S Z A P C R R R R R R R R
Operands	**Clocks**	**Transfers***	**Bytes**	**Coding Example**
(no operands)	24	3	1	IRET

JA/JNBE	JA/JNBE short-label Jump if above/Jump if not below nor equal			Flags O D I T S Z A P C
Operands	**Clocks**	**Transfers***	**Bytes**	**Coding Example**
short-label	16 or 4	—	2	JA ABOVE

JAE/JNB	JAE/JNB short-label Jump if above or equal/Jump if not below			Flags O D I T S Z A P C
Operands	**Clocks**	**Transfers***	**Bytes**	**Coding Example**
short-label	16 or 4	—	2	JAE ABOVE_EQUAL

JB/JNAE	JB/JNAE short-label Jump if below/Jump if not above nor equal			Flags O D I T S Z A P C
Operands	**Clocks**	**Transfers***	**Bytes**	**Coding Example**
short-label	16 or 4	—	2	JB BELOW

JBE/JNA	JBE/JNA short-label Jump if below or equal/Jump if not above			Flags O D I T S Z A P C
Operands	**Clocks**	**Transfers***	**Bytes**	**Coding Example**
short-label	16 or 4	—	2	JNA NOT_ABOVE

JC	JC short-label Jump if carry			Flags O D I T S Z A P C
Operands	**Clocks**	**Transfers***	**Bytes**	**Coding Example**
short-label	16 or 4	—	2	JC CARRY_SET

JCXZ	JCXZ short-label Jump if CX is zero			Flags O D I T S Z A P C
Operands	**Clocks**	**Transfers***	**Bytes**	**Coding Example**
short-label	18 or 6	—	2	JCXZ COUNT_DONE

JE/JZ	JE/JZ short-label Jump if equal/Jump if zero			Flags O D I T S Z A P C
Operands	**Clocks**	**Transfers***	**Bytes**	**Coding Example**
short-label	16 or 4	—	2	JZ ZERO

JG/JNLE	JG/JNLE short-label Jump if greater/Jump if not less nor equal			Flags O D I T S Z A P C
Operands	**Clocks**	**Transfers***	**Bytes**	**Coding Example**
short-label	16 or 4	—	2	JG GREATER

JGE/JNL	JGE/JNL short-label Jump if greater or equal/Jump if not less			Flags O D I T S Z A P C
Operands	**Clocks**	**Transfers***	**Bytes**	**Coding Example**
short-label	16 or 4	—	2	JGE GREATER_EQUAL

JL/JNGE	JL/JNGE short-label Jump if less/Jump if not greater nor equal			Flags O D I T S Z A P C
Operands	**Clocks**	**Transfers***	**Bytes**	**Coding Example**
short-label	16 or 4	—	2	JL LESS

JLE/JNG	JLE/JNG short-label Jump if less or equal/Jump if not greater			Flags O D I T S Z A P C
Operands	**Clocks**	**Transfers***	**Bytes**	**Coding Example**
short-label	16 or 4	—	2	JNG NOT_GREATER

JMP	JMP target Jump			Flags O D I T S Z A P C
Operands	**Clocks**	**Transfers***	**Bytes**	**Coding Example**
short-label	15	—	2	JMP SHORT
near-label	15	—	3	JMP WITHIN_SEGMENT
far-label	15	—	5	JMP FAR_LABEL
memptr16	18+EA	1	2-4	JMP [BX].TARGET
regptr16	11	—	2	JMP CX
memptr32	24+EA	2	2-4	JMP OTHER.SEG [SI]

JNC	JNC short-label Jump if not carry			Flags O D I T S Z A P C
Operands	**Clocks**	**Transfers***	**Bytes**	**Coding Example**
short-label	16 or 4	—	2	JNC NOT_CARRY

JNE/JNZ	JNE/JNZ short-label Jump if not equal/Jump if not zero			Flags O D I T S Z A P C
Operands	**Clocks**	**Transfers***	**Bytes**	**Coding Example**
short-label	16 or 4	—	2	JNE NOT_EQUAL

JNO	JNO short-label Jump if not overflow			Flags O D I T S Z A P C
Operands	**Clocks**	**Transfers***	**Bytes**	**Coding Example**
short-label	16 or 4	—	2	JNO NO_OVERFLOW

JNP/JPO

JNP/JPO short-label
Jump if not parity/Jump if parity odd

Flags: O D I T S Z A P C

Operands	Clocks	Transfers*	Bytes	Coding Example
short-label	16 or 4	—	2	JPO ODD_PARITY

JNS

JNS short-label
Jump if not sign

Flags: O D I T S Z A P C

Operands	Clocks	Transfers*	Bytes	Coding Example
short-label	16 or 4	—	2	JNS POSITIVE

JO

JO short-label
Jump if overflow

Flags: O D I T S Z A P C

Operands	Clocks	Transfers*	Bytes	Coding Example
short-label	16 or 4	—	2	JO SIGNED_OVRFLW

JP/JPE

JP/JPE short-label
Jump if parity/Jump if parity even

Flags: O D I T S Z A P C

Operands	Clocks	Transfers*	Bytes	Coding Example
short-label	16 or 4	—	2	JPE EVEN_PARITY

JS

JS short-label
Jump if sign

Flags: O D I T S Z A P C

Operands	Clocks	Transfers*	Bytes	Coding Example
short-label	16 or 4	—	2	JS NEGATIVE

LAHF

LAHF (no operands)
Load AH from flags

Flags: O D I T S Z A P C

Operands	Clocks	Transfers*	Bytes	Coding Example
(no operands)	4	—	1	LAHF

LDS

LDS destination,source
Load pointer using DS

Flags: O D I T S Z A P C

Operands	Clocks	Transfers	Bytes	Coding Example
reg16, mem32	16+EA	2	2-4	LDS SI,DATA.SEG [DI]

LEA

LEA destination,source
Load effective address

Flags: O D I T S Z A P C

Operands	Clocks	Transfers*	Bytes	Coding Example
reg16, mem16	2+EA	—	2-4	LEA BX, [BP] [DI]

LES

LES destination,source
Load pointer using ES

Flags: O D I T S Z A P C

Operands	Clocks	Transfers*	Bytes	Coding Example
reg16, mem32	16+EA	2	2-4	LES DI, [BX].TEXT_BUFF

LOCK

LOCK (no operands)
Lock bus

Flags: O D I T S Z A P C

Operands	Clocks	Transfers*	Bytes	Coding Example
(no operands)	2	—	1	LOCK XCHG FLAG,AL

LODS

LODS source-string
Load string

Flags: O D I T S Z A P C

Operands	Clocks	Transfers*	Bytes	Coding Example
source-string	12	1	1	LODS CUSTOMER_NAME
(repeat) source-string	9+13/rep	1/rep	1	REP LODS NAME

LOOP

LOOP short-label
Loop

Flags: O D I T S Z A P C

Operands	Clocks	Transfers*	Bytes	Coding Example
short-label	17/5	—	2	LOOP AGAIN

LOOPE/LOOPZ

LOOPE/LOOPZ short-label
Loop if equal/Loop if zero

Flags: O D I T S Z A P C

Operands	Clocks	Transfers*	Bytes	Coding Example
short-label	18 or 6	—	2	LOOPE AGAIN

LOOPNE/LOOPNZ

LOOPNE/LOOPNZ short-label
Loop if not equal/Loop if not zero

Flags: O D I T S Z A P C

Operands	Clocks	Transfers*	Bytes	Coding Example
short-label	19 or 5	—	2	LOOPNE AGAIN

NMI†

NMI (external nonmaskable interrupt)
Interrupt if NMI = 1

Flags: O S I T S Z A P C
0 0

Operands	Clocks	Transfers*	Bytes	Coding Example
(no operands)	50	5	N/A	N/A

MOV

MOV destination,source
Move

Flags: O D I T S Z A P C

Operands	Clocks	Transfers*	Bytes	Coding Example
memory, accumulator	10	1	3	MOV ARRAY [SI], AL
accumulator, memory	10	1	3	MOV AX, TEMP_RESULT
register, register	2	—	2	MOV AX,CX
register, memory	8+EA	1	2-4	MOV BP, STACK_TOP
memory, register	9+EA	1	2-4	MOV COUNT [DI], CX
register, immediate	4	—	2-3	MOV CL, 2
memory, immediate	10+EA	1	3-6	MOV MASK [BX] [SI], 2CH
seg-reg, reg16	2	—	2	MOV ES, CX
seg-reg, mem16	8+EA	1	2-4	MOV DS, SEGMENT_BASE
reg16, seg-reg	2	—	2	MOV BP, SS
memory, seg-reg	9+EA	1	2-4	MOV [BX].SEG_SAVE, CS

MOVS

MOVS dest-string,source-string
Move string

Flags: O D I T S Z A P C

Operands	Clocks	Transfers*	Bytes	Coding Example
dest-string, source-string	18	2	1	MOVS LINE EDIT_DATA
(repeat) dest-string, source-string	9+17/rep	2/rep	1	REP MOVS SCREEN, BUFFER

MOVSB/MOVSW

MOVSB/MOVSW (no operands)
Move string (byte/word)

Flags: O D I T S Z A P C

Operands	Clocks	Transfers*	Bytes	Coding Example
(no operands)	18	2	1	MOVSB
(repeat) (no operands)	9+17/rep	2/rep	1	REP MOVSW

MUL

MUL source
Multiplication, unsigned

Flags: O D I T S Z A P C
X U U U U X

Operands	Clocks	Transfers*	Bytes	Coding Example
reg8	70-77	—	2	MUL BL
reg16	118-133	—	2	MUL CX
mem8	(76-83) + EA	1	2-4	MUL MONTH [SI]
mem16	(124-139) + EA	1	2-4	MUL BAUD_RATE

NEG

NEG destination
Negate

Flags: O D I T S Z A P C
X X X X X 1*

Operands	Clocks	Transfers*	Bytes	Coding Example
register	3	—	2	NEG AL
memory	16 + EA	2	2-4	NEG MULTIPLIER

*0 if destination = 0

NOP

NOP (no operands)
No Operation

Flags: O D I T S Z A P C

Operands	Clocks	Transfers*	Bytes	Coding Example
(no operands)	3	—	1	NOP

NOT

NOT destination
Logical not

Flags: O D I T S Z A P C

Operands	Clocks	Transfers*	Bytes	Coding Example
register	3	—	2	NOT AX
memory	16 + EA	2	2-4	NOT CHARACTER

OR

OR destination,source
Logical inclusive or

Flags: O D I T S Z A P C
0 X X U X 0

Operands	Clocks	Transfers*	Bytes	Coding Example
register, register	3	—	2	OR AL, BL
register, memory	9 + EA	1	2-4	OR DX, PORT_ID [DI]
memory, register	16 + EA	2	2-4	OR FLAG_BYTE, CL
accumulator, immediate	4	—	2-3	OR AL, 01101100B
register, immediate	4	—	3-4	OR CX,01H
memory, immediate	17 + EA	2	3-6	OR [BX].CMD_WORD,0CFH

OUT

OUT port,accumulator
Output byte or word

Flags: O D I T S Z A P C

Operands	Clocks	Transfers*	Bytes	Coding Example
immed8, accumulator	10	1	2	OUT 44, AX
DX, accumulator	8	1	1	OUT DX, AL

POP

POP destination
Pop word off stack

Flags: O D I T S Z A P C

Operands	Clocks	Transfers*	Bytes	Coding Example
register	8	1	1	POP DX
seg-reg (CS illegal)	8	1	1	POP DS
memory	17 + EA	2	2-4	POP PARAMETER

POPF

POPF (no operands)
Pop flags off stack

Flags: O D I T S Z A P C
R R R R R R R R

Operands	Clocks	Transfers*	Bytes	Coding Example
(no operands)	8	1	1	POPF

PUSH

PUSH source
Push word onto stack

Flags: O D I T S Z A P C

Operands	Clocks	Transfers*	Bytes	Coding Example
register	11	1	1	PUSH SI
seg-reg (CS legal)	10	1	1	PUSH ES
memory	16 + EA	2	2-4	PUSH RETURN_CODE [SI]

PUSHF

PUSHF (no operands)
Push flags onto stack

Flags: O D I T S Z A P C

Operands	Clocks	Transfers*	Bytes	Coding Example
(no operands)	10	1	1	PUSHF

RCL

RCL destination,count
Rotate left through carry

Flags: O D I T S Z A P C
X X

Operands	Clocks	Transfers*	Bytes	Coding Example
register, 1	2	—	2	RCL CX, 1
register, CL	8 + 4/bit	—	2	RCL AL, CL
memory, 1	15 + EA	2	2-4	RCL ALPHA, 1
memory, CL	20 + EA + 4/bit	2	2-4	RCL [BP].PARM, CL

RCR

RCR designation,count
Rotate right through carry

Flags: O D I T S Z A P C
X X

Operands	Clocks	Transfers*	Bytes	Coding Example
register, 1	2	—	2	RCR BX, 1
register, CL	8 + 4/bit	—	2	RCR BL, CL
memory, 1	15 + EA	2	2-4	RCR [BX].STATUS, 1
memory, CL	20 + EA + 4/bit	2	2-4	RCR ARRAY [DI], CL

REP

REP (no operands)
Repeat string operation

Flags: O D I T S Z A P C

Operands	Clocks	Transfers*	Bytes	Coding Example
(no operands)	2	—	1	REP MOVS DEST, SRCE

REPE/REPZ

REPE/REPZ (no operands)
Repeat string operation while equal/while zero

Flags: O D I T S Z A P C

Operands	Clocks	Transfers*	Bytes	Coding Example
(no operands)	2	—	1	REPE CMPS DATA, KEY

REPNE/REPNZ

REPNE/REPNZ (no operands)
Repeat string operation while not equal/not zero

Flags: O D I T S Z A P C

Operands	Clocks	Transfers*	Bytes	Coding Example
(no operands)	2	—	1	REPNE SCAS INPUT_LINE

RET

RET optional-pop-value
Return from procedure

Flags: O D I T S Z A P C

Operands	Clocks	Transfers*	Bytes	Coding Example
(intra-segment, no pop)	8	1	1	RET
(intra-segment, pop)	12	1	3	RET 4
(inter-segment, no pop)	18	2	1	RET
(inter-segment, pop)	17	2	3	RET 2

ROL

ROL destination,count
Rotate left

Flags: O D I T S Z A P C
X X

Operands	Clocks	Transfers	Bytes	Coding Examples
register, 1	2	—	2	ROL BX, 1
register, CL	8 + 4/bit	—	2	ROL DI, CL
memory, 1	15 + EA	2	2-4	ROL FLAG BYTE [DI],1
memory, CL	20 + EA + 4/bit	2	2-4	ROL ALPHA , CL

ROR

ROR destination,count
Rotate right

Flags: O D I T S Z A P C
X X

Operand	Clocks	Transfers*	Bytes	Coding Example
register, 1	2	—	2	ROR AL, 1
register, CL	8 + 4/bit	—	2	ROR BX, CL
memory, 1	15 + EA	2	2-4	ROR PORT STATUS, 1
memory, CL	20 + EA + 4/bit	2	2-4	ROR CMD WORD, CL

SAHF

SAHF (no operands)
Store AH into flags

Flags: O D I T S Z A P C
R R R R R

Operands	Clocks	Transfers*	Bytes	Coding Example
(no operands)	4	—	1	SAHF

SAL/SHL

SAL/SHL destination,count
Shift arithmetic left/Shift logical left

Flags: O D I T S Z A P C
X X

Operands	Clocks	Transfers*	Bytes	Coding Examples
register,1	2	—	2	SAL AL,1
register, CL	8 + 4/bit	—	2	SHL DI, CL
memory,1	15 + EA	2	2-4	SHL [BX].OVERDRAW, 1
memory, CL	20 + EA + 4/bit	2	2-4	SAL STORE COUNT, CL

SAR

SAR destination,source
Shift arithmetic right

Flags: O D I T S Z A P C
X X X U X X

Operands	Clocks	Transfers*	Bytes	Coding Example
register, 1	2	—	2	SAR DX, 1
register, CL	8 + 4/bit	—	2	SAR DI, CL
memory, 1	15 + EA	2	2-4	SAR N BLOCKS, 1
memory, CL	20 + EA + 4/bit	2	2-4	SAR N BLOCKS, CL

SBB

SBB destination,source
Subtract with borrow

Flags: O D I T S Z A P C
X X X X X X

Operands	Clocks	Transfers*	Bytes	Coding Example
register, register	3	—	2	SBB BX, CX
register, memory	9 + EA	1	2-4	SBB DI, [BX].PAYMENT
memory, register	16 + EA	2	2-4	SBB BALANCE, AX
accumulator, immediate	4	—	2-3	SBB AX, 2
register, immediate	4	—	3-4	SBB CL, 1
memory, immediate	17 + EA	2	3-6	SBB COUNT [SI], 10

SCAS

SCAS dest-string
Scan string

Flags: O D I T S Z A P C
X X X X X X

Operands	Clocks	Transfers*	Bytes	Coding Example
dest-string	15	1	1	SCAS INPUT LINE
(repeat) dest-string	9 + 15/rep	1/rep	1	REPNE SCAS BUFFER

SEGMENT†

SEGMENT override prefix
Override to specified segment

Flags: O D I T S Z A P C

Operands	Clocks	Transfers*	Bytes	Coding Example
(no operands)	2	—	1	MOV SS:PARAMETER, AX

SHR

SHR destination,count
Shift logical right

Flags: O D I T S Z A P C
X X

Operands	Clocks	Transfers*	Bytes	Coding Example
register, 1	2	—	2	SHR SI, 1
register, CL	8 + 4/bit	—	2	SHR SI, CL
memory, 1	15 + EA	2	2-4	SHR ID BYTE [SI] [BX], 1
memory, CL	20 + EA + 4/bit	2	2-4	SHR INPUT WORD, CL

SINGLE STEP†

SINGLE STEP (Trap flag interrupt)
Interrupt if TF = 1

Flags: O D I T S Z A P C
0 0

Operands	Clocks	Transfers*	Bytes	Coding Example
(no operands)	50	5	N/A	N/A

*For the 8086, add four clocks for each 16-bit word transfer with an odd address. For the 8088, add four clocks for each 16-bit word transfer.

†ASM-86 incorporates the segment override prefix into the operand specification and not as a separate instruction. SEGMENT is included only for timing information.

†SINGLE STEP is not an instruction, it is included only for timing information.

STC

STC (no operands)
Set carry flag

Flags: O D I T S Z A P C
1

Operands	Clocks	Transfers*	Bytes	Coding Example
(no operands)	2	—	1	STC

STD

STD (no operands)
Set direction flag

Flags: O D I T S Z A P C
1

Operands	Clocks	Transfers*	Bytes	Coding Example
(no operands)	2	—	1	STD

STI

STI (no operands)
Set interrupt enable flag

Flags: O D I T S Z A P C
1

Operands	Clocks	Transfers*	Bytes	Coding Example
(no operands)	2	—	1	STI

STOS

STOS dest-string
Store byte or word string

Flags: O D I T S Z A P C

Operands	Clocks	Transfers*	Bytes	Coding Example
dest-string	11	1	1	STOS PRINT LINE
(repeat) dest-string	9+10/rep	1/rep	1	REP STOS DISPLAY

SUB

SUB destination,source
Subtraction

Flags: O D I T S Z A P C
X X X X X X

Operands	Clocks	Transfers*	Bytes	Coding Example
register, register	3	—	2	SUB CX, BX
register, memory	9+EA	1	2-4	SUB DX, MATH_TOTAL [SI]
memory, register	16+EA	2	2-4	SUB [BP+2], CL
accumulator, immediate	4	—	2-3	SUB AL, 10
register, immediate	4	—	3-4	SUB SI, 5280
memory, immediate	17+EA	2	3-6	SUB [BP].BALANCE, 1000

TEST

TEST destination,source
Test or non-destructive logical and

Flags: O D I T S Z A P C
0 X X U X 0

Operands	Clocks	Transfers*	Bytes	Coding Example
register, register	3	—	2	TEST SI, DI
register, memory	9+EA	1	2-4	TEST SI, END_COUNT
accumulator, immediate	4	—	2-3	TEST AL, 00100000B
register, immediate	5	—	3-4	TEST BX, 0CC4H
memory, immediate	11+EA	—	3-6	TEST RETURN_CODE, 01H

WAIT

WAIT (no operands)
Wait while $\overline{\text{TEST}}$ pin not asserted

Flags: O D I T S Z A P C

Operands	Clocks	Transfers*	Bytes	Coding Example
(no operands)	3 + 5n	—	1	WAIT

XCHG

XCHG destination,source
Exchange

Flags: O D I T S Z A P C

Operands	Clocks	Transfers*	Bytes	Coding Example
accumulator, reg16	3	—	1	XCHG AX, BX
memory, register	17+EA	2	2-4	XCHG SEMAPHORE, AX
register, register	4	—	2	XCHG AL, BL

XLAT

XLAT source-table
Translate

Flags: O D I T S Z A P C

Operands	Clocks	Transfers*	Bytes	Coding Example
source-table	11	1	1	XLAT ASCII_TAB

XOR

XOR destination,source
Logical exclusive or

Flags: O D I T S Z A P C
0 X X U X 0

Operands	Clocks	Transfers*	Bytes	Coding Example
register, register	3	—	2	XOR CX, BX
register, memory	9+EA	1	2-4	XOR CL, MASK_BYTE
memory, register	16+EA	2	2-4	XOR ALPHA [SI], DX
accumulator, immediate	4	—	2-3	XOR AL, 01000010B
register, immediate	4	—	3-4	XOR SI, 00C2H
memory, immediate	17+EA	2	3-6	XOR RETURN_CODE, 0D2H

DICTIONNAIRE DES INSTRUCTIONS

AAA *Ajustement ASCII Addition*
AAD *Ajustement ASCII Division*
AAM *Ajustement ASCII Multiplication*
AAS *Ajustement ASCII Soustraction*
ADC *Addition avec retenue*
ADD *Addition sans retenue*
AND *« ET » logique*
CALL *Appel sous-programme intra et intersegments*
CBW *Conversion d'un octet dans un mot*
CLC *Effacement de la retenue indicateur CF*
CLD *Incrémentation automatique des opérations de chaîne*
CLI *Suppression des interruptions externes*
CMC *Complément de la retenue indicateur CF*
CMP *Comparaison de deux opérandes*
CMPS *Comparaison chaîne de caractères*
CWD *Conversion d'un mot dans un double mot*
DAA *Ajustement décimal addition*
DAS *Ajustement décimal soustraction*
DEC *Décrémentation par 1*
DIV *Division non signée*
ESC *Instruction coprocesseur « escape »*
HLT *Mise en veille de l'unité centrale*
IDIV *Division signée*
IMUL *Multiplication signée*
IN *Instruction lecture de port d'entrée/sortie*
INC *Incrémentation par 1*

INT *Appel au superviseur interruption logicielle*
INTO *Appel au superviseur si dépassement de capacité*
IRET *Retour de service d'interruption*
Jcond *Débranchement conditionnel*
JMP *Branchement inconditionnel intra et intersegments*
LAHF *Chargement des indicateurs dans le registre AH*
LDS *Chargement registre DS et registre avec une adresse logique*
LEA *Chargement registre avec une adresse effective*
LES *Chargement registre ES et registre avec une adresse logique*
LODS *Chargement chaîne de caractères*
LOOP *Contrôle de boucle inconditionnel*
LOOPcond *Contrôle de boucle conditionnel*
MOV *Mouvement de données (copie)*
MOVS *Mouvement chaîne de caractères*
MUL *Multiplication non signée*
NEG *Complément à deux (négation)*
NOP *Instruction nulle (non-opération)*
NOT *Opération logique « NON »*
OR *« OU » logique*
OUT *Instruction d'écriture sur port d'entrée/sortie*
POP *Dépilage d'un élément à partir de la pile*
POPF *Dépilage du registre des indicateurs*
PUSH *Empilage d'un élément dans la pile*
PUSHF *Empilage du registre des indicateurs*
RCL *Rotation à gauche avec prise de la retenue*
RCR *Rotation à droite avec prise de la retenue*
REP *Répétition chaîne de caractères*
REPcond *Répétition conditionnelle chaîne de caractères*
RET *Retour de procédure (sous-programme)*
ROL *Rotation à gauche sans prise de la retenue*
ROR *Rotation à droite sans prise de la retenue*
SAHF *Transfert du registre AH dans le registre d'indicateurs*
SAL *Décalage à gauche avec prise de la retenue*
SAR *Décalage à droite avec prise de la retenue*
SBB *Soustraction avec retenue*
SCAS *Recherche chaîne de caractères*
SHL *Décalage à gauche sans prise de la retenue*
SHR *Rotation à droite sans prise de la retenue*
STC *Initialisation de la retenue indicateur CF*
STD *Décrémentation automatique des opérations de chaîne*
STI *Autorisation des interruptions externes*
STOS *Stockage chaîne de caractères*
SUB *Soustraction sans retenue*
TEST *Comparaison logique de bits*
WAIT *Attente de fin d'exécution du coprocesseur*
XCHG *Échange registre avec registre ou mémoire*
XLAT *Translation avec table*
XOR *Opération logique « OU » exclusif*

Matrice du jeu d'instruction du 8086

Hi \ Lo	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	ADD b.f.r/m	ADD w.f.r/m	ADD b.t.r/m	ADD w.t.r/m	ADD b.ia	ADD w.ia	PUSH ES	POP ES	OR b.f.r/m	OR w.f.r/m	OR b.t.r/m	OR w.t.r/m	OR b.i	OR w.i	PUSH CS	2 Byte
1	ADC b.f.r/m	ADC w.f.r/m	ADC b.t.r/m	ADC w.t.r/m	ADC b.i	ADC w.i	PUSH SS	POP SS	SBB b.f.r/m	SBB w.f.r/m	SBB b.t.r/m	SBB w.t.r/m	SBB b.i	SBB w.i	PUSH DS	POP DS
2	AND b.f.r/m	AND w.f.r/m	AND b.t.r/m	AND w.t.r/m	AND b.i	AND w.i	SEG ES	DAA	SUB b.f.r/m	SUB w.f.r/m	SUB b.t.r/m	SUB w.t.r/m	SUB b.i	SUB w.i	SEG CS	DAS
3	XOR b.f.r/m	XOR w.f.r/m	XOR b.t.r/m	XOR w.t.r/m	XOR b.i	XOR w.i	SEG SS	AAA	CMP b.f.r/m	CMP w.f.r/m	CMP b.t.r/m	CMP w.t.r/m	CMP b.i	CMP w.i	SEG DS	AAS
4	INC AX	INC CX	INC DX	INC BX	INC SP	INC BP	INC SI	INC DI	DEC AX	DEC CX	DEC DX	DEC BX	DEC SP	DEC BP	DEC SI	DEC DI
5	PUSH AX	PUSH CX	PUSH DX	PUSH BX	PUSH SP	PUSH BP	PUSH SI	PUSH DI	POP AX	POP CX	POP DX	POP BX	POP SP	POP BP	POP SI	POP DI
6	PUSHA	POPA	BOUND r.r/m	ARPL r.r/m					PUSH iw	IMUL r.iw.r/m	PUSH is	IMUL r.is.r/m	INS b	INS w	OUTS b	OUTS w
7	JO	JNO	JB/ JNAE	JNB/ JAE	JE/ JZ	JNE/ JNZ	JBE/ JNA	JNBE/ JA	JS	JNS	JP/ JPE	JNP/ JPO	JL/ JNGE	JNL/ JGE	JLE/ JNG	JNLE/ JG
8	Immed b.r/m	Immed w.r/m	Immed b.r/m	Immed is.r/m	TEST b.r/m	TEST w.r/m	XCHG b.r/m	XCHG w.r/m	MOV b.f.r/m	MOV w.f.r/m	MOV b.t.r/m	MOV w.t.r/m	MOV sr.t.r/m	LEA	MOV sr.t.r/m	POP r/m
9	NOP	XCHG CX	XCHG DX	XCHG BX	XCHG SP	XCHG BP	XCHG SI	XCHG DI	CBW	CWD	CALL i.d	WAIT	PUSHF	POPF	SAHF	LAHF
A	MOV m → AL	MOV m → AX	MOV AL → m	MOV AX → m	MOVS b	MOVS w	CMPS b	CMPS w	TEST b.i	TEST w.i	STOS b	STOS w	LODS b	LODS w	SCAS b	SCAS w
B	MOV i → AL	MOV i → CL	MOV i → DL	MOV i → BL	MOV i → AH	MOV i → CH	MOV i → DH	MOV i → BH	MOV i → AX	MOV i → CX	MOV i → DX	MOV i → BX	MOV i → SP	MOV i → BP	MOV i → SI	MOV i → DI
C	Shift b.r/m.i	Shift w.r/m.i	RET (i-SP)	RET	LES	LDS	MOV b.i.r/m	MOV w.i.r/m	ENTER iw.ib	LEAVE	RET I.(i-SP)	RET I	INT Type 3	INT (Any)	INTO	IRET
D	Shift b	Shift w	Shift b.v	Shift w.v	AAM	ADD		XLAT	ESC 0	ESC 1	ESC 2	ESC 3	ESC 4	ESC 5	ESC 6	ESC 7
E	LOOPNZ/ LOOPNE	LOOPZ/ LOOPE	LOOP	JCXZ	IN b	IN w	OUT b.	OUT w	CALL d	JMP d	JMP i.d	JMP si.d	IN v.b	IN v.w	OUT v.d	OUT v.w
F	LOCK		REP	REP Z	HLT	CMC	Grp 1 b.r/m	Grp 1 w.r/m	CLC	STC	CLI	STI	CLD	STD	Grp 2 b.r/m	Grp 2 w.r/m

where

mod ... r/m	000	001	010	011	100	101	110	111
Immed	ADD	OR	ADC	SBB	AND	SUB	XOR	CMP
Shift	ROL	ROR	RCL	RCR	SHL/SAL	SHR	—	SAR
Grp 1	TEST	—	NOT	NEG	MUL	IMUL	DIV	IDIV
Grp 2	INC	DEC	CALL id	CALL I.id	JMP id	JMP I.id	PUSH	—
2 Byte 0F 00	SLDT	STR	LLDT	LTR	VERR	VERW	—	—
2 Byte 0F 01	SGDT	SIDT	LGDT	LIDT	SMSW	LMSW	—	—
2 Byte, second is			LAR r.r/m	LSL r.r/m	—	—	CLTS	—

b = byte operation
d = direct
f = from CPU reg
i = immediate
ia = immed. to accum.
ib = immediate byte
id = indirect
is = immed. byte sign ext.
iw = immediate word
I = long ie. intersegment
m = memory
r = register
r/m = EA is second byte
si = short intrasegment
sr = segment register
t = to CPU reg
v = variable
w = word operation
z = zero

ASSIGNATION DES CHAMPS MODRM

CHAMP reg :

reg	reg16	reg8
000	AX	AL
001	CX	CL
010	DX	DL
011	BX	BL

reg	reg16	reg8
100	SP	AH
101	BP	CH
110	SI	DH
111	DI	BH

CHAMP rs :

rs	rseg
00	ES
01	CS

rs	rseg
10	SS
11	DS

CHAMP mod et r/m :

mod r/m	EA	rseg
00 000	[BX + SI]	DS
00 001	[BX + DI]	DS
00 010	[BP + SI]	SS
00 011	[BP + DI]	SS
00 100	[SI]	DS
00 101	[DI]	DS
00 110	adrs16	DS
00 111	[BX]	DS
01 000	[BX + SI + depl8]	DS
01 001	[BX + DI + depl8]	DS
01 010	[BP + SI + depl8]	SS
01 011	[BP + DI + depl8]	SS
01 100	[SI + depl8]	DS
01 101	[DI + depl8]	DS
01 110	[BP + depl8]	SS
01 111	[BX + depl8]	DS

mod r/m	EA	rseg
10 000	[BX + SI + depl16]	DS
10 001	[BX + DI + depl16]	DS
10 010	[BP + SI + depl16]	SS
10 011	[BP + DI + depl16]	SS
10 100	[SI + depl16]	DS
10 101	[DI + depl16]	DS
10 110	[BP + depl16]	SS
10 111	[BX + depl16]	DS
11 000	AX / AL	
11 001	CX / CL	
11 010	DX / DL	
11 011	BX / BL	
11 100	SP / AH	
11 101	BP / CH	
11 110	SI / DH	
11 111	DI / BH	

Jcond

A pour « au-dessus »,
B pour « en dessous »,
C pour « retenue »,
E pour « égal, pair »,
G pour « plus grand »,
L pour « plus petit »,
N pour « non, pas »,
O pour « dépassement de capacité, impair »,
P pour « parité »,
S pour « signé »,
Z pour « zéro ».

Les symboles valides sont les suivants :

JA	JNA	JO	JNO
JAE	JNAE	JP	JNP
JB	JNB	JPE	JPO
JBE	JNBE	JS	JNS
JC	JNC	JZ	JNZ
JE	JNE		
JG	JNG		
JGE	JNGE		
JL	JNL		
JLE	JNLE		

LANGAGE MACRO

;; *Commentaire macro double point-virgule* .....
! *Délimiteur de chaîne de caractères macro* .....
% *Symbole de conversion numérique d'une chaîne de caractères macro* .....
& *Symbole d'expansion d'une variable macro* .....
= *Symbole d'affectation d'une variable macro* .....
ENDM *Délimiteur de macro* .....
EXITM *Sortie inconditionnelle d'une macro-instruction* .....
IF *Test macro si non zéro* .....
IFE *Test macro si zéro* .....
IF1 *Test macro si première passe* .....
IF2 *Test macro si deuxième passe* .....
IFDEF *Test macro si variable définie* .....
IFNDEF *Test macro si variable non définie* .....
IFB *Test macro si paramètre omis* .....
IFNB *Test macro si paramètre défini* .....
IFDIF *Test macro si paramètre différent* .....
IFIDN *Test macro si paramètre identique* .....
INCLUDE *Chargement global d'une bibliothèque* .....
IRP *Répétition macro à partir d'une liste d'arguments* .....
IRPC *Répétition macro à partir d'une liste de caractères* .....
LOCAL *Définition d'étiquettes uniques dans une macro* .....
MACRO *Prototype macro* .....
PURGE *Élimination de macros après chargement* .....
REPT *Répétition macro d'un paragraphe macro* .....

OPÉRATEURS ET ATTRIBUTS

Notation *Notation numérique dans les expressions* .....
Précédence *Ordre des précédences dans les expressions* .....
AND *Opération « ET »* .....
Arithmétique *Opérations arithmétiques* .....
MOD *Modulus* .....
NOT *Opération « NON »* .....
Or *Opération « OU »* .....
Relationnel *Opérations relationnelles* .....
SHL *Décalage logique à gauche* .....
SHR *Décalage logique à droite* .....
XOR *Opération « OU EXCLUSIF »* .....
= *Affectation ou équivalence* .....
PTR *Modification de type* .....
Segment *Modification de registre de segment* .....
SHORT *Modification d'adresse de débranchement* .....
HIGH *Identification de position de poids fort* .....
LOW *Identification de position de poids faible* .....
THIS *Identification de position du compteur d'assemblage* .....
LENGHT *Valeur de duplication d'une définition* .....
MASK *Valeur du masque d'isolation d'un champ binaire* .....
OFFSET *Valeur de déplacement dans un segment (adresse effective)* .....
SEG *Valeur de paragraphe d'une variable* .....
SHIFT *Valeur de décalage pour isoler un champ binaire* .....
SIZE *Valeur d'encombrement d'une définition* .....
TYPE *Valeur du type d'une définition* .....
WIDTH *Valeur de longueur d'un champ binaire* .....

DIRECTIVES

Directives .....
Format .....
Syntaxe .....
= *Affectation ou équivalence* .....
DB *Définition de donnée (octet)* .....
DD *Définition de donnée (double mot)* .....
DQ *Définition de donnée (quatre mots)* .....
DT *Définition de donnée (dix octets)* .....
DW *Définition de donnée (un mot)* .....
EQU *Équivalence* .....
LABEL *Définition d'un label* .....
RECORD *Définition chaîne de bits* .....
STRUC *Définition d'une structure de données* .....
XXXXX (RECORD) *Allocation chaîne de bits* .....
XXXX (STRUC) *Allocation structure de données* .....
ASSUME *Déclaration des segments* .....
COMMENT *Définition de commentaires* .....
END *Délimiteur fin de source* .....
EXTRN *Définition de référence externe* .....
GROUP *Définition de groupe de segments* .....
EVEN *Alignement de code* .....
NAME *Identification du programme* .....
ORG *Alignement du compteur d'assemblage* .....
PROC *Définition d'une procédure* .....
PUBLIC *Déclaration de référence* .....
.RADIX *Déclaration de base numérique* .....
SEGMENT *Définition de segment* .....
.CREF et .XCREF *Gestion de la liste des résultats* .....
.LALL, .SALL et .XALL *Gestion du type des résultats* .....
.LIST et .XLIST *Gestion de la liste des résultats* .....
%OUT *Gestion de la progression de l'assemblage* .....
PAGE *Gestion de la mise en page des résultats* .....
SUBTTL *Sous-titre pour la liste des résultats* .....
TITLE *Titre des en-têtes de liste des résultats* .....

Les directives de l'assembleur sont des commandes exécutées au moment de l'assemblage et non pas au moment du fonctionnement du programme. Elles ne génèrent pas d'instructions pour l'unité centrale. Elles créent des constantes, des zones de mémoire, ou donnent à l'assembleur des informations sur le programme. Elles peuvent définir des portions du programme qui ne seront pas assemblées, des macros, afficher des informations à l'écran pendant l'assemblage. En voici la liste.

AND	ET logique.
ASSUME	Désigne les registres de segments.
BYTE	Définit des données de type octet.
COMMENT	Bloc de commentaires.
.CREF	Liste de symboles pour référence croisée.
DB	Définit une zone de stockage de type octet.
DD	Définit une zone de stockage de type double mot, soit quatre octets.
DQ	Définit une zone de stockage de type quadruple mot, soit huit octets.
DT	Définit une zone de dix octets.
DW	Définit une zone de type mot.
DWORD	Définit une zone de type double mot (32 bits).
DUP	Duplique une chaîne.
ELSE	Seconde alternative d'un bloc conditionnel.
END	Fin du programme source.
ENDIF	Fin d'un bloc conditionnel.
ENDM	Fin de macro.
ENDP	Fin de procédure.
ENDS	Fin de segment ou de structure.
EQ	Égalité logique.
EQU	Assigne une constante.
EVEN	Initialise une adresse paire.
EXISTM	Sortie d'une macro.
EXTRN	Définition des références externes.
FAR	Désigne une étiquette en dehors du segment.
GE	Plus grand ou égal logique.
GROUP	Associe les segments avec un nom de groupe.
GT	Plus grand que logique.
HIGH	Moitié haute d'un mot.
IF	Bloc conditionnel.
IF1	Conditionnel première passe.
IF2	Conditionnel seconde passe.
IFB	Conditionnel si paramètre manquant.
IFDEF	Conditionnel si défini.
IFDIF	Conditionnel si deux paramètres différents.
IFE	Conditionnel si zéro.
IFIDN	Conditionnel si deux paramètres identiques.
IFNB	Conditionnel si paramètre non manquant.
INCLUDE	Insertion de code source à partir d'un autre fichier.
IRP	Macro de répétition indéfinie.
IRPC	Macro de répétition indéfinie d'un caractère.
LABEL	Création d'une étiquette.
LE	Inférieur ou égal logique.
LENGHT	Détermination de la longueur d'un symbole.
LOCAL	Définition locale d'un symbole dans une macro.
LOW	Partie basse d'un mot.
LT	Plus petit que logique.
.LALL	Liste des instructions de macros.
.LFCOND	Liste des blocs conditionnels.
.LIST	Liste du programme.
MACRO	Définition d'une macro.
NAME	Définition du nom du programme.
MASK	Définition d'un schéma de masque.
MOD	Modulo.
NAME	Définition d'un nom de module.
NE	Non-égalité logique.
NEAR	Désigne une étiquette dans le segment.
NOT	Complément logique.
OFFSET	Déplacement dans un segment.
OR	Opération logique OU.
ORG	Initialisation du pointeur d'instructions.
%OUT	Affichage d'une chaîne sur l'écran vidéo.
PAGE	Début d'une nouvelle page et détermination de sa taille.
PROC	Début d'une procédure.
PTR	Utilisé avec BYTE, WORD, DWORD, NEAR et FAR pour spécifier la taille d'une étiquette ou d'une variable.
PUBLIC	Définition d'un symbole global.
.RADIX	Choix d'une base.
RECORD	Définition d'un type d'enregistrement.
REPT	Macro de répétition.
.SALL	Suppression du listing des macros.
SEG	Valeur de segment associé à un symbole.
SEGMENT	Définition d'un segment.
SHL	Décalage gauche logique.
SHORT	Saut court en avant.
SHR	Décalage droit logique.
SIZE	Détermination de la taille d'une étiquette.
.SFCOND	Suppression du listing des conditionnels faux.
STRUC	Définition de structures.
SUBTTL	Définition de sous-titres.
.TCOND	Listing des blocs conditionnels par défaut.
TITLE	Définition du titre du programme.
XOR	Opération logique OU exclusif.
.XALL	Liste des macros qui créent du code ou des données.
.XCREF	Suppression du listing de référence croisée.
.XLIST	Suppression du listing du programme.
WIDTH	Détermination de la largeur d'un champ.
WORD	Définit une donnée de type mot.
=	Assigne une valeur à un symbole.
$	Déplacement courant.

Les directives de l'Assembleur.

INTERRUPTIONS LOGICIELLES

INT 20H *Fin de programme* .....
INT 21H *Appel MS-DOS (DOS-CALL)* .....
INT 22H *Point de terminaison de programme* .....
INT 23H *Point d'arrêt en urgence (CNTRL-BREAK EXIT)* .....
INT 24H *Point d'arrêt en cas d'erreur (CRITICAL ERROR EXIT)* .....
INT 25H *Lecture directe sur disque avec adresse absolue* .....
INT 26H *Écriture directe sur disque avec adresse absolue* .....
INT 27H *Mise en mode résident d'un programme* .....
INT 28H et INT 3FH *Interruptions utilisées par les internes du DOS* .....

FONCTIONS INT 21H (DOS CALL)

Définition .....
FONCT 0 *Fin de programme* .....
FONCT 1 *Entrée au clavier* .....
FONCT 2 *Affichage à l'écran* .....
FONCT 3 *Lecture port série auxiliaire* .....
FONCT 4 *Écriture port série auxiliaire* .....
FONCT 5 *Écriture port imprimante* .....
FONCT 6 *Lecture et écriture combinées sur clavier écran* .....
FONCT 7 *Lecture directe au clavier sans écho* .....
FONCT 8 *Entrée au clavier sans écho* .....
FONCT 9 *Envoi d'un message entier à l'écran* .....
FONCT 0AH *Lecture d'un message entier au clavier* .....
FONCT 0BH *Test d'entrée au clavier d'un caractère* .....
FONCT 0CH *Effacement de l'entrée d'une réponse au clavier* .....
FONCT 0DH *Restauration de l'environnement des fichiers disques* .....
FONCT 0EH *Affectation de l'unité de disque par défaut* .....
FONCT 0FH *Ouverture d'un fichier sur disque* .....
FONCT 10H *Fermeture d'un fichier sur disque* .....
FONCT 11H *Recherche d'un fichier sur disque* .....
FONCT 12H *Recherche du fichier suivant sur disque* .....
FONCT 13H *Suppression d'un fichier sur disque* .....
FONCT 14H *Lecture séquentielle* .....
FONCT 15H *Écriture séquentielle* .....
FONCT 16H *Création d'un fichier sur disque* .....
FONCT 17H *Changement de nom d'un fichier sur disque* .....
FONCT 18H *Fonction interne du DOS* .....
FONCT 19H *Recherche de l'unité de disque courante* .....
FONCT 1AH *Affectation de la DTA* (Disk Transfer Address) .....
FONCT 1BH *Information sur la FAT* (File Allocation Table) .....
FONCT 1CH *Information spécifique sur la FAT* (File Allocation Table) .....
FONCT 1DH à FONCT 20H *Fonctions internes du DOS* .....
FONCT 21H *Lecture directe* .....
FONCT 22H *Écriture directe* .....
FONCT 23H *Encombrement d'un fichier sur disque* .....
FONCT 24H *Calcul de l'adresse directe sur disque* .....
FONCT 25H *Affectation d'un vecteur d'interruption* .....
FONCT 26H *Création d'un nouveau PSP* (Program Segment Prefix) .....
FONCT 27H *Lecture directe par blocs* .....
FONCT 28H *Écriture directe par blocs* .....
FONCT 29H *Analyse de nom de fichier* .....
FONCT 2AH *Lecture de la date* .....
FONCT 2BH *Initialisation de la date* .....
FONCT 2CH *Lecture de l'heure* .....
FONCT 2DH *Initialisation de l'heure* .....
FONCT 2EH *Bascule de vérification d'écriture* .....
FONCT 2FH *Chargement de l'adresse de la DTA* .....
FONCT 30H *Chargement de la version de MS-DOS* .....
FONCT 31H *Fin de traitement et mise en mode résident* .....
FONCT 32H *Fonction interne du DOS* .....
FONCT 33H *Contrôle de l'arrivée de CNTRL-BREAK* .....
FONCT 34H *Fonction interne du DOS* .....
FONCT 35H *Lecture d'un vecteur d'interruption* .....
FONCT 36H *Récupération de l'espace libre sur disque* .....
FONCT 37H *Fonction interne du DOS* .....
FONCT 38H *Récupération des informations internationales dépendantes des pays.*
FONCT 39H *Création d'un sous-directory sur disque* .....
FONCT 3AH *Suppression d'une entrée du directory* .....
FONCT 3BH *Attribution du sous-directory courant* .....
FONCT 3CH *Création d'un fichier sur disque* .....
FONCT 3DH *Ouverture d'un fichier* .....
FONCT 3EH *Fermeture d'un fichier* .....
FONCT 3FH *Lecture d'un enregistrement à partir d'un fichier* .....
FONCT 40H *Écriture d'un enregistrement dans un fichier* .....
FONCT 41H *Suppression d'un fichier dans un directory étendu* .....
FONCT 42H *Déplacement du pointeur de lecture ou d'écriture dans un fichier*
FONCT 43H *Changement de mode d'un fichier dans un directory étendu* .....
FONCT 44H *Manipulations multiples pour les unités d'entrées/sorties* .....
FONCT 45H *Duplication d'un pointeur de manipulation de fichier* .....
FONCT 46H *Duplication forcée d'un pointeur de manipulation de fichier* .....
FONCT 47H *Lecture du nom du directory courant* .....
FONCT 48H *Allocation mémoire* .....
FONCT 49H *Libération mémoire* .....
FONCT 4AH *Modification d'une allocation de mémoire* .....
FONCT 4BH *Chargement et exécution d'un programme* .....
FONCT 4CH *Fin de programme avec code retour* .....
FONCT 4DH *Récupération du code retour d'une sous-tâche* .....
FONCT 4EH *Recherche du premier fichier correspondant* .....
FONCT 4FH *Recherche du fichier suivant sur disque* .....
FONCT 50H à FONCT 53H *Fonctions internes du DOS* .....
FONCT 54H *Test si vérification d'écriture* .....
FONCT 55H *Fonction interne du DOS* .....
FONCT 56H *Renomination d'un fichier* .....
FONCT 57H *Manipulation de la date et de l'heure dans un fichier* .....

Les vecteurs d'interruption BIOS sont des adresses sur quatre bits qui font référence à des sous-programmes placés dans le BIOS ROM. On peut y accéder à l'aide de l'instruction

INT X

où X est le numéro de vecteur.

Vecteur	Adresse	Fonction
5	14	Copie d'écran
8	20	Horloge
9	24	Scan code clavier
10	40	Écran vidéo
11	44	Octet de configuration
12	48	Taille mémoire
13	4C	Service disque
14	50	Transfert série
15	54	Cassette et TopView
16	58	Caractères clavier
17	5C	Imprimante
18	60	BASIC ROM
19	64	Chargeur
1A	68	Heure
1B	6C	Break clavier
1C	70	Horloge

Les vecteurs d'interruption faisant appel à des fonctions BIOS

Les vecteurs d'interruption DOS sont des adresses sur quatre bits qui font référence à des sous-programmes se trouvant dans le DOS. On peut y accéder à l'aide de l'instruction

INT X

où X est le numéro de vecteur.

Vecteur	Adresse	Fonction
20	80	Fin d'un programme COM
21	84	Fonctions générales DOS
22	88	Code de terminaison d'un programme
23	8C	Code d'interruption clavier
24	90	Code d'erreur
25	94	Lecture absolue sur disque
26	98	Écriture absolue sur disque
27	9C	Fin d'un programme restant résident

Les vecteurs d'interruption faisant appel à des fonctions DOS

Les fonctions de l'interruption 21 hex permettent d'effectuer des opérations courantes d'entrée et de sortie. On y accède en plaçant le numéro de fonction dans le registre AH et en exécutant l'interruption 21 hex.

AH	Fonction
0	Fin.
1	Entrée clavier.
2	Affichage d'un caractère.
3	Entrée port série.
4	Sortie port série.
5	Sortie imprimante.
6	Entrée directe au clavier, affichage direct.
7	Entrée directe au clavier sans affichage.
8	Entrée au clavier, pas de tabulation, pas d'affichage.
9	Affichage d'une chaîne.
A	Lecture du buffer clavier.
B	Lecture de l'état du clavier.
25	Modification d'un vecteur interruption.
30	Obtention du numéro de version DOS.
31	Fin d'un programme restant résident.
35	Obtention d'un vecteur interruption.
3C	Création d'un fichier.
3D	Ouverture d'un fichier.
3E	Fermeture d'un fichier.
3F	Lecture d'un fichier ou d'un périphérique.
40	Écriture d'un fichier ou d'un périphérique.
41	Effacement d'un fichier.
43	Détermination ou modification des attributs d'un fichier.
4A	Modification de la mémoire allouée.
4B	Chargement et exécution d'un programme.
4C	Fin d'un programme.
4D	Détermination d'un code d'erreur.
56	Changement de nom d'un fichier.

Les fonctions DOS de l'interruption 21H.

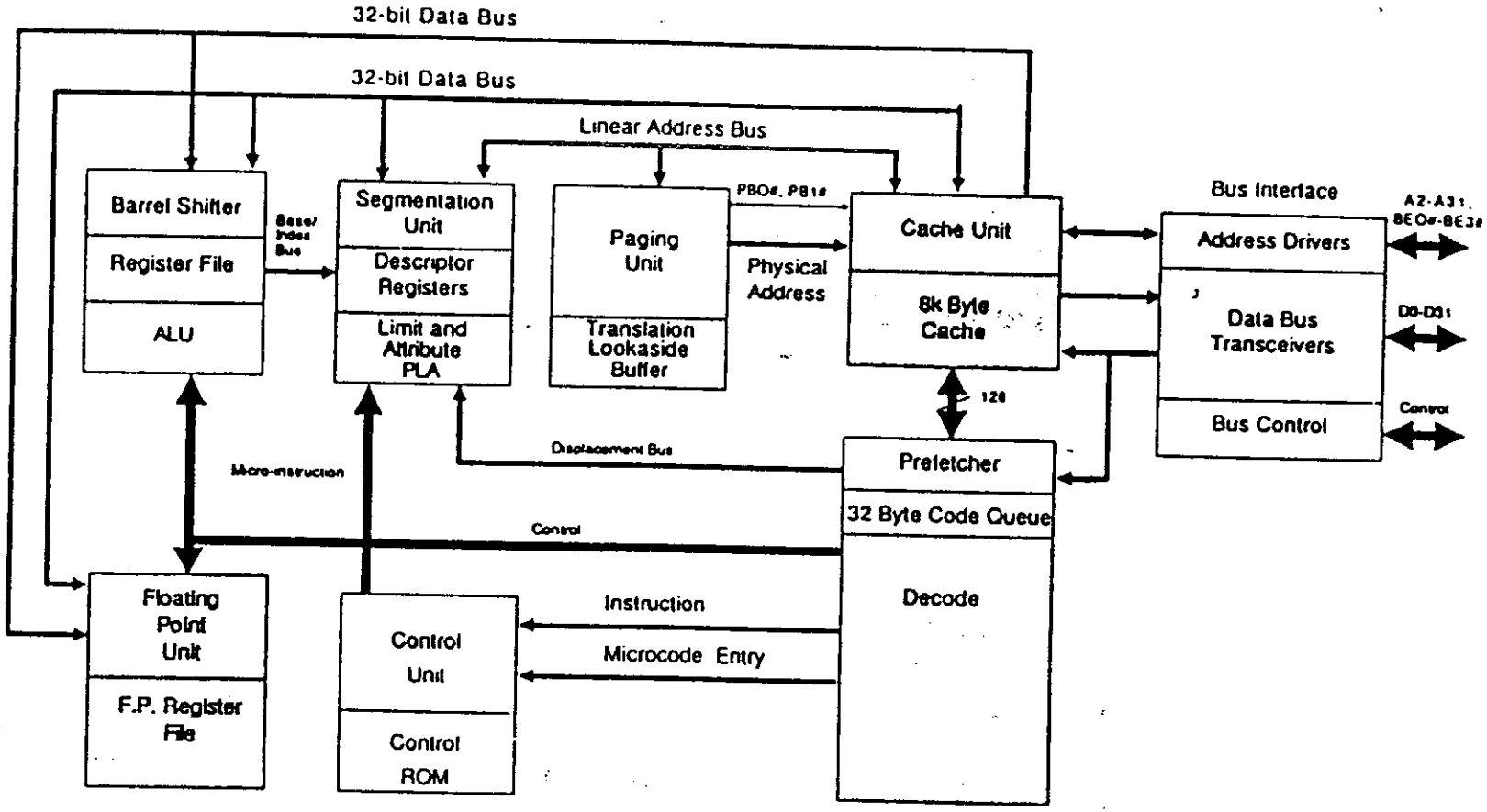
Ø	Fin de programme : identique à INT 2ØH.
1	Entrée au clavier : attend un caractère, l'affiche à l'écran et le retourne dans AL. Si on a frappé Ctrl+Break, déclenche INT 23H.
2	Affichage à l'écran du caractère dans DL
3	Lit un caractère sur la carte de transmission asynchrone et le range dans AL.
4	Envoie le caractère dans DL à la carte de transmission asynchrone.
5	Imprime (imprimante) le caractère contenu dans DL.
6	Entrées-sorties directes. Si (DL)=ØFFH, AL reçoit un caractère du clavier s'il en existe. Si (DL) est différent de ØFFH, DL contient un caractère envoyé à l'écran.
7	Entrée directe : attend qu'un caractère soit frappé (à moins qu'il n'y en ait un de prêt) et l'envoie dans AL, sans affichage. Pas de vérification de la frappe de Ctrl+Break.
8	Entrée du clavier, comme (1), mais sans affichage.
9	Chaîne de caractères : DS:DX doit pointer une chaîne de caractères en mémoire, se terminant par le symbole $ (soit 24H). Chaque caractère de la chaîne est affiché (comme en 2).
A	Entrée du clavier vers la mémoire tampon.
B	Vérification de l'état de l'unité d'entrée.
C	Vidage du tampon du clavier, puis exécution de l'une des fonctions 1, 6, 7, 8 et A dont le numéro est dans AL.
2D	Initialisation de l'heure.
2E	Commutateur de vérification.
2F	Obtention de la zone DTA.
3Ø	Initialisation du numéro de version du DOS.
31	Fin de traitement en restant résidant.
32	Vérification de Ctrl+Break.
34	Utilisé par le DOS en interne.
35	Obtention d'un vecteur.
36	Obtention d'espace libre sur disque.
37	Utilisé par le DOS en interne.

AH	Fonction
38	Retour d'information en fonction du pays.
39	Création d'un sous-répertoire (MD).
3A	Suppression d'un répertoire (RD).
3B	Changement de répertoire (CD).
3C	Création d'un fichier (CREAT).
3D	Ouverture d'un fichier.
3E	Fermeture d'un code d'identité de fichier.
3F	Lecture à partir d'un fichier ou d'une unité.
4Ø	Ecriture dans un fichier ou une unité.
41	Suppression d'un fichier.
42	Déplacement du pointeur de fichier.
43	Changement de mode de fichier.
44	Contrôle d'entrées-sorties pour les unités.
45	Duplication d'un code d'identité de fichier.
46	Duplication forcée d'un code d'identité.
47	Obtention du répertoire courant.
48	Affectation de la mémoire.
49	Libération de la mémoire attribuée.
4A	Modification de blocs de mémoire attribués.
4B	Chargement ou exécution d'un programme.
4C	Fin d'un traitement (EXIT) et retour au DOS : à utiliser à la place de INT 2ØH.

Commande	Syntaxe	Fonction
Assemble	A [adresse]	Assemble en mémoire les instructions assembleur.
Compare	C < bloc > < adresse >	Compare deux zones mémoire.
Dump	D [bloc]	Affiche le contenu d'une zone mémoire.
Enter	E < adresse > [suite]	Entre des données en mémoire.
Fill	F < bloc > < suite >	Remplit une zone mémoire.
GO	G [= adresse] [adresse]	Exécute le programme se trouvant en mémoire. Permet jusqu'à 10 points d'arrêt.
Hexa	H <valeur > < valeur >	Additionne et soustrait deux valeurs en hexadécimal.
Input	I < valeur >	Lit et affiche le contenu d'un octet sur un port d'E/S.
Load	L [adresse [unité secteur valeur]]	Charge un fichier disque en mémoire.
Move	M < bloc > < adresse >	Transfère le contenu d'une zone mémoire dans une autre.
Name	N < Nomfichier. type > [Nomfichier.type]	Définit le nom de fichier pour les commandes L et W.
Output	0 < octet > < valeur >	Envoie un octet sur un port E/S.
Quit	Q	Permet de sortir de DEBUG.
Registers	R [registre]	Affiche et/ou modifie des registres et des indicateurs.
Search	S < bloc > < suite >	Recherche d'une suite d'octets dans une zone mémoire.
Trace	T [= adresse] [valeur]	Exécution d'instructions pas à pas avec affichage des registres, indicateurs et instructions.
Unassemble	U [bloc]	Affiche les instructions en langage source avec les adresses et les valeurs d'octets.
Write	W [adresse [unité secteur valeur]]	Sauvegarde sur disque le fichier mis au point.

DECIMAL VALUE ➡		0	16	32	48	64	80	96	112	128	144	160	176	192	208	224	240
⬇	HEXA DECIMAL VALUE	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	0	BLANK (NULL)	►	BLANK (SPACE)	0	@	P	‘	p	Ç	É	á	▒	╚	╩	∝	≡
1	1	☺	◄	!	1	A	Q	a	q	ü	æ	í	▒	╠	╦	β	±
2	2	☻	↕	"	2	B	R	b	r	é	Æ	ó	▓	╩	╥	Γ	≥
3	3	♥	‼	#	3	C	S	c	s	â	ô	ú	│	╠	╙	π	≤
4	4	♦	¶	$	4	D	T	d	t	ä	ö	ñ	┤	═	╘	Σ	⌠
5	5	♣	§	%	5	E	U	e	u	à	ò	Ñ	╡	╬	╒	σ	⌡
6	6	♠	▬	&	6	F	V	f	v	å	û	ª	╢	╣	╓	µ	÷
7	7	•	↨	'	7	G	W	g	w	ç	ù	º	╖	╦	╫	τ	≈
8	8	◘	↑	(	8	H	X	h	x	ê	ÿ	¿	╕	╠	╪	Φ	°
9	9	○	↓	)	9	I	Y	i	y	ë	Ö	⌐	╣	╩	┘	Θ	∙
10	A	◙	→	*	:	J	Z	j	z	è	Ü	¬	║	╦	│	Ω	·
11	B	♂	←	+	;	K	[	k	{	ï	¢	½	╗	╠	■	δ	√
12	C	♀	∟	,	<	L	\	l	¦	î	£	¼	╝	╬	■	∞	n
13	D	♪	↔	—	=	M	]	m	}	ì	¥	¡	╜	╩	■	φ	2
14	E	♫	▲	.	>	N	^	n	~	Ä	₧	«	╛	╬	■	∈	▮
15	F	☼	▼	/	?	O	_	o	△	Å	ƒ	»	┘	╩	■	∩	BLANK FF

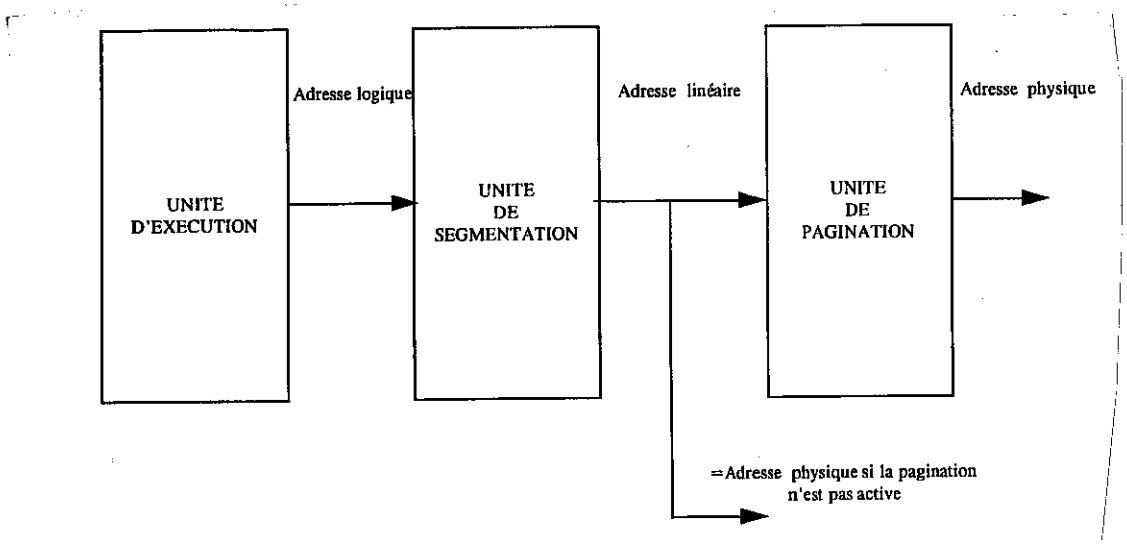
i486 BLOCK DIAGRAM



NEW INTEL386 ARCHITECTURE IMPLEMENTATION

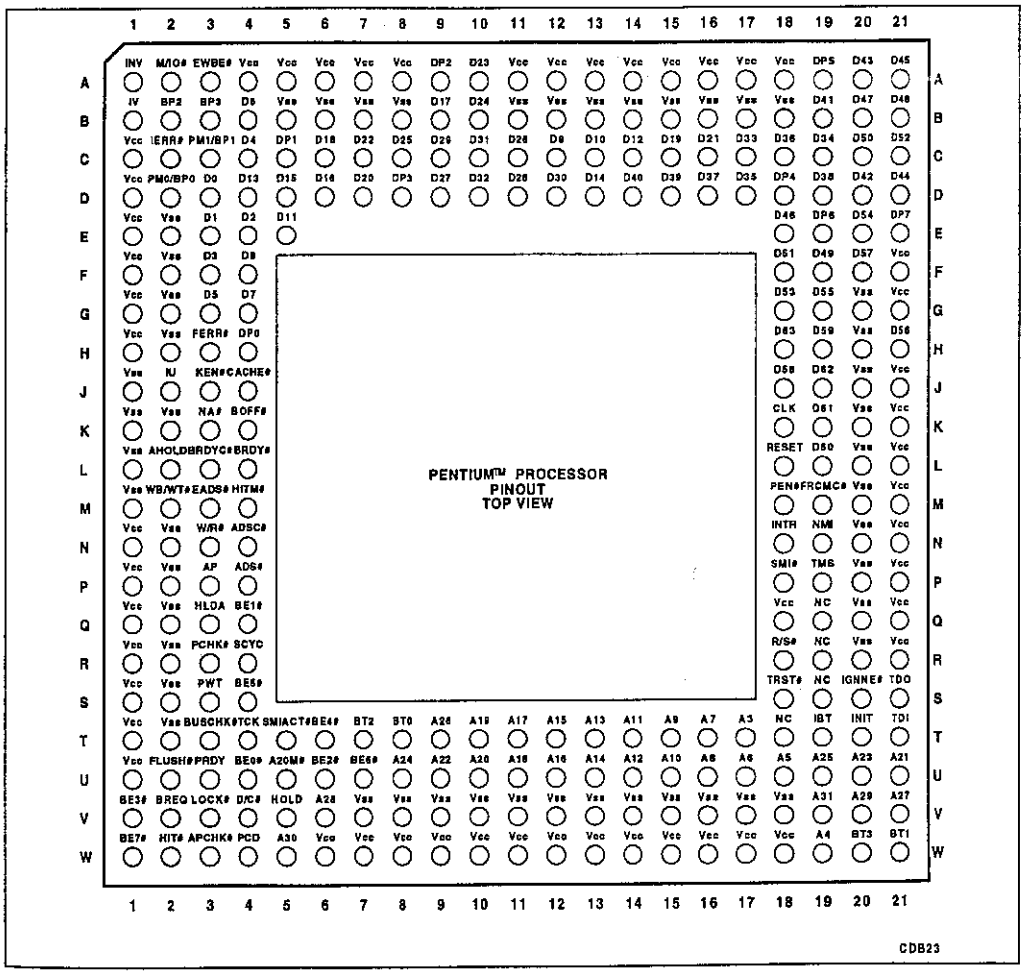
UNITE A VIRGULE FLOTTANTE
64-BITS
BTB
ROM CONTROLE
32
32-BITS
V
UNITE CONTROLE
ALU
32
REGISTRES
Unité de Segmentation
Registres de segment
Unité Interface de Bus
BUFFERS de PREFETCH
ALU
256 BITS
U
32
32
CACHE CODE
TLB
CACHE DONNEES
TLB
Unité de pagination
Unité d'exécution
32
64
64
64
ADRESSE
64
DONNEE
CONTROLE

Architecture interne du Pentium



Générations des différentes adresses

Pentium™ Processor Pinouts



EAX		AH AX AL
EBX		BH BX BL
ECX		CH CX CL
EDX		DH DX DL
ESI		SI
EDI		DI
EBP		BP
ESP		SP

...téraux

CS
DS
SS
ES
FS
GS

Registres de segment

EIP		IP

Registre pointeur d'instruction

EFLAGS		FLAGS

Registre d'état EFLAGS